Mehmet TAŞOĞLU
20210601059
Software Engineering

Table Of Contents:

1- INTRODUCTION

2- PROCEDURAL CONTENT GENERATION (PCG)

3- AI AGENT IMPLEMENTATION

4- ENEMY SYSTEM

5- USER INTERFACE AND GAME FLOW

6- CONCLUSION

1. INTRODUCTION

1.1 Project Overview:

For this project, I built a 2D dungeon generation system that creates levels procedurally and includes an AI-controlled character that moves through these dungeons. My implementation relies on Binary Space Partitioning (BSP) to create the dungeon layout and Wave Function Collapse (WFC) to add visual elements.

1.2 Project Goals:

In this project, my goal was to create 100×100 tile-based dungeons with multiple chambers and connecting passages, develop two different dungeon themes using Wave Function Collapse (WFC), build an AI character with pathfinding capabilities using the A* algorithm and behavior logic based on a Finite State Machine (FSM), place a total of ten enemy units consisting of three stationary, four patrolling, and three chasing enemies, and ensure that my AI agent could successfully navigate to the objective without being eliminated.

2. PROCEDURAL CONTENT GENERATION (PCG)

2.1 BSP Algorithm Implementation:

2.1.1 Dungeon Structure:

In my BSP implementation, I divided the 100×100 game area into progressively smaller sections by applying five levels of recursive division, randomly choosing between horizontal and vertical cuts. During this process, I ensured that the smallest chambers were 6×6 tiles in size, while the largest chambers reached up to 15×15 tiles.

```
[Header("Dungeon Settings")]
public int dungeonWidth = 100;
public int dungeonHeight = 100;
public int minRoomWidth = 6;
public int minRoomHeight = 6;
public int maxRoomWidth = 15;
public int maxRoomHeight = 15;

[Range(3, 8)]
public int maxDepth = 5;
```

In my project, I used multiple Tilemaps to separate and organize dungeon elements efficiently. The floorTilemap represents walkable areas, the wallsTilemap defines impassable boundaries, the objectsTilemap contains interactive or blocking elements, and the decorTilemap is used for visual details that do not affect gameplay. Additionally, the debugTilemap was utilized during development to visualize debugging and system-related information.

2.1.2 Room Generation:

In my system, I created one chamber within each final partition of the BSP tree, ensuring that each chamber was smaller than its containing area and positioned randomly within it. I then connected all chambers to one another through a system of passages to maintain full navigability throughout the dungeon.

2.1.3 Corridor System:

In my design, I created L-shaped passages that link the center points of chambers by moving either horizontally first and then vertically, or following the opposite sequence, which guarantees accessibility to all chambers. I used doorway tiles to clearly mark the transitions between chambers and passages. This BSP-based approach worked well, maintained full connectivity between all chambers, and completed the generation process in under 0.5 seconds.

However, I encountered some issues during development. Passages occasionally intersected with each other, which I resolved by applying proper sequencing, and my initial chamber dimensions were insufficient, leading me to adjust the boundary values accordingly.

2.2 Wave Function Collapse (WFC):

2.2.1 Template Design:

I designed 2 distinct dungeon using different visual elements.

Template 1: Medieval Dungeon Template

Template 2: Dangerous Cave Template

2.2.2 WFC Implementation:

My WFC algorithm applies decoration patterns to chambers while preserving navigation routes.

```
[Header("WFC Settings")]
[Tooltip("Minimum distance from doors")]
[Range(1, 5)]
public int doorClearance = 2;

[Tooltip("Minimum path width to maintain")]
[Range(1, 3)]
public int minPathWidth = 2;

private System.Random random;
private HashSet<Vector2Int> protectedTiles = new HashSet<Vector2Int>();
private List<DungeonTemplate> availableTemplates = new List<DungeonTemplate>();
```

```
void SetupProtectedTiles(RectInt room, List<Vector2Int> doorPositions, List<Vector2Int> corridors)
{
    protectedTiles.Clear();

    foreach (Vector2Int door in doorPositions)
    {
        if (room.Contains(door))
        {
            AddProtectedArea(door, doorClearance);
        }
    }

    foreach (Vector2Int corridor in corridors)
    {
        if (room.Contains(corridor))
        {
            AddProtectedArea(corridor, minPathWidth);
        }
    }
}
```

2.2.3 Tile Types Created:

Pillar

Decoration_Wall

Rubble

Obstacle

Trap

3. AI AGENT IMPLEMENTATION

3.1 Pathfinding Algorithm:

3.1.1 A* Implementation:

```
private class Node
{
    public Vector2Int position;
    public Node parent;
    public float gCost;
    public float hCost;
    public float fCost => gCost + hCost;

    public Node(Vector2Int pos)
    {
        position = pos;
    }
}
```

Path Calculation Process:

1. Initialize starting position with zero g-cost
2. Select node with lowest f-cost from open list
3. Explore all 8 neighboring positions
4. Update path costs when finding better routes
5. Reconstruct final path by tracing parent nodes backwards

3.1.2 Walkability Validation:

My system determines tile walkability through multiple checks.

In my system, I defined specific walkability requirements to ensure correct navigation. A tile is considered walkable if a floor tile exists at that position or if the tile is marked as a door tile, while the presence of a wall tile at the same position makes it non-walkable. Door tiles are treated as traversable elements to preserve connectivity between chambers and passages.

3.2 Decision-Making System:

3.2.1 Finite State Machine (FSM):

My AI uses a 4-state FSM for behavior control.

```
private enum AIState
{
    Moving,
    Evading,
    Attacking,
    ReachedGoal
}
```

Moving → Evading (strong enemy within 12 tiles)

Moving → Attacking (weak enemy within 8 tiles)

Evading → Moving (threat eliminated OR distance > 6 tiles OR timeout after 3s)

Attacking → Moving (enemy dead OR distance > 10 tiles)

Any State → ReachedGoal (when reaching goal position)

State Update Methods:

- UpdateMoving(): Path following and periodic recalculation
- UpdateEvading(): Escape movement with alternative angle attempts
- UpdateAttacking(): Combat decision between melee and ranged
- CheckForEnemies(): Continuous threat monitoring in all active states

3.2.2 Enemy Classification:

My threat assessment combines enemy health and distance.

```
// Weak Enemy: health < 60 HP
EnemyBase FindNearestWeakEnemy()
{
    if (enemy.health < 60f)  // Attack threshold
        return enemy;
}

// Strong Enemy: health >= 70 HP
EnemyBase FindNearestThreat()
{
    if (enemy.health >= strongEnemyThreshold)  // Evade threshold
        return enemy;
}
```

8 tiles: Weak enemy detection range (initiate attack)

12 tiles: Strong enemy detection range (initiate evade)

6 tiles: Safe distance threshold (stop evading)

10 tiles: Combat disengagement distance (return to goal)

3.2.3 Combat System:

My agent employs dual combat modes based on distance.

Melee Combat (Distance < 2 tiles):

```
void MeleeAttack(EnemyBase enemy)
{
    if (enemy == null) return;

    enemy.TakeDamage(attackDamage);
    Debug.Log($"× "" attack on {enemy.enemyType} for {attackDamage} damage!");

    if (spriteRenderer != null)
        StartCoroutine(FlashWhite());
}
```

Ranged Combat (Distance 2-8 tiles):

```
    void ShootProjectile(Vector3 direction, EnemyBase target)
{
    PlayerShooting shooting = GetComponent<PlayerShooting>();

    if (shooting != null && shooting.HasAmmo())
    {
        Vector3 targetWorldPos = target.transform.position;
        Vector3 screenPos = Camera.main.WorldToScreenPoint(targetWorldPos);

        shooting.ShootTowards(direction);

        lastAttackTime = Time.time;

        Debug.Log($"🔫 Shot at {target.enemyType}!");

        if (spriteRenderer != null)
            StartCoroutine(FlashWhite());
    }
    else
    {
        Debug.LogWarning("⚠ Out of ammo! Switching to melee.");
        MeleeAttack(target);
    }
}
```

Combat State Behavior:

- Distance > 10 tiles: Disengage and return to Moving state
- Distance 2-8 tiles: Use ranged attacks with projectiles
- Distance < 2 tiles: Switch to melee attacks
- Enemy becomes strong (> 70 HP): Automatically switch to Evading

4. ENEMY SYSTEM

4.1 Enemy Types:

Stationary Enemies:

I designed three stationary enemies that remain at their spawn locations and continuously orient themselves toward the player character. These enemies engage the player when they enter a range of seven tiles.

Patrol Enemies:

I implemented four patrol enemies that move back and forth within predefined patrol zones. When the player enters their proximity within these boundaries, they pursue the player and resume their patrol pattern once the pursuit ends.

Chaser Enemies:

I created three chaser enemies that continuously track the player's position and actively pursue them within a range of fifteen tiles. If the player escapes this range, the enemies return to their original positions.

4.2 Enemy Spawning:

My enemy spawning system follows strict placement rules to ensure fair and functional gameplay. Enemies are positioned at least fifteen tiles away from the starting chamber, at least ten tiles away from the objective chamber, and maintain a minimum distance of three tiles from each other.

5. User Interface and Game Flow

In my project, I implemented a Demo UI Manager to control the main menu, gameplay interface, and overall level flow. The system handles transitions between the menu and gameplay, manages game time, and allows different dungeon levels to be generated using predefined templates with specific seed and room size values. During gameplay, the UI displays real-time player information such as time, health, ammo, remaining enemies, and distance to the goal. It also handles key game events like player death and goal completion, provides on-screen feedback, and allows the player to restart the level, return to the main menu, or quit the game.

The Problem of Creation Same Dungeon and Same Position of Enemy:

In my DemoUIManager script, I encountered an issue where generating Level 1 or any other level always produced the same dungeon layout, and enemy units always spawned at the same positions. This behavior occurs because the random number generator is initialized with a fixed seed value before the dungeon generation process begins. As a result, the procedural generation becomes deterministic, meaning that the same input always produces the same output.

Since both dungeon generation and enemy placement rely on Unity's Random class, resetting the random seed causes all randomly generated elements, including room placement, corridor paths, and enemy spawn positions, to remain identical every time the same level is generated.

Code Location Causing the Issue:

The issue originates from the GenerateLevel(int templateIndex) method in the DemoUIManager script.

```
Random.InitState(template.seed != 0 ? template.seed : (int)System.DateTime.Now.Ticks);
```

By calling Random.InitState() with a fixed seed value, I reset Unity's random number generator to the same initial state every time a level is generated. When the same seed is used, all random operations occur in the same order, which leads to identical dungeon layouts and enemy spawn positions. This directly affects systems such as BSP dungeon partitioning, room placement, corridor generation, and enemy spawning logic.

The seed value is defined inside the dungeon template structure.

```
[System.Serializable]
public class DungeonTemplate
{
    public string name;
    public int seed;
    public int minRoomSize = 6;
    public int maxRoomSize = 15;
}
```

Since each dungeon template can have a predefined seed value, selecting the same level template always applies the same seed.

Enemy spawn positions are also affected because they are generated after the random seed is initialized. As enemy placement uses random position selection within the dungeon layout, resetting the seed causes enemies to spawn at the same coordinates in every playthrough of the same level.

6. Conclusion

What Needs Improvement?
I would improve the project by as well as collectible items and health restoration mechanics.I would also implement adaptive difficulty scaling to better match the player's performance and design a minimap interface to improve navigation. In addition, I would expand the AI agent's decision-making system by making its attack and escape behaviors more detailed and dynamic. Finally, I would modify the level generation system so that each time a generate level button is pressed, a completely different dungeon is created, along with varied enemy spawn positions based on the new layout.

Final Thoughts:

Through this project, I learned that using a BSP approach provides reliable and well-structured dungeon layouts, while Wave Function Collapse (WFC) adds visual variety without increasing system complexity. I also observed that the A* algorithm is highly efficient for grid-based pathfinding, and that Finite State Machines (FSM) work well for implementing clear and simple AI behaviors. Additionally, integrating vitality values and different behavior types significantly improves the perceived intelligence and realism of AI agents.My project successfully integrates PCG and AI methodologies. My agent navigates intricate dungeons while making informed combat decisions. The system maintains flexibility and produces diverse, playable environments.